Edit

Add Comments and Docstrings by Using Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo, an organization that needs to improve the documentation of a new Python utility module. In this Challenge Lab, you will use GitHub Copilot to automatically generate docstrings for several functions. Next, you will leverage Copilot Chat to add clarifying inline comments to a more complex function. Finally, you will complete the documentation by adding a module-level docstring and then verifying all of your work using Python's built-in pydoc utility.

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md

▶ Quick tips for navigating the Challenge Lab instructions.

New to Challenge Labs? Click here to learn more.

Generate function docstrings with Copilot

Hints Enabled

No Yes

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using T Passw0rd! as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

  GitHub Username

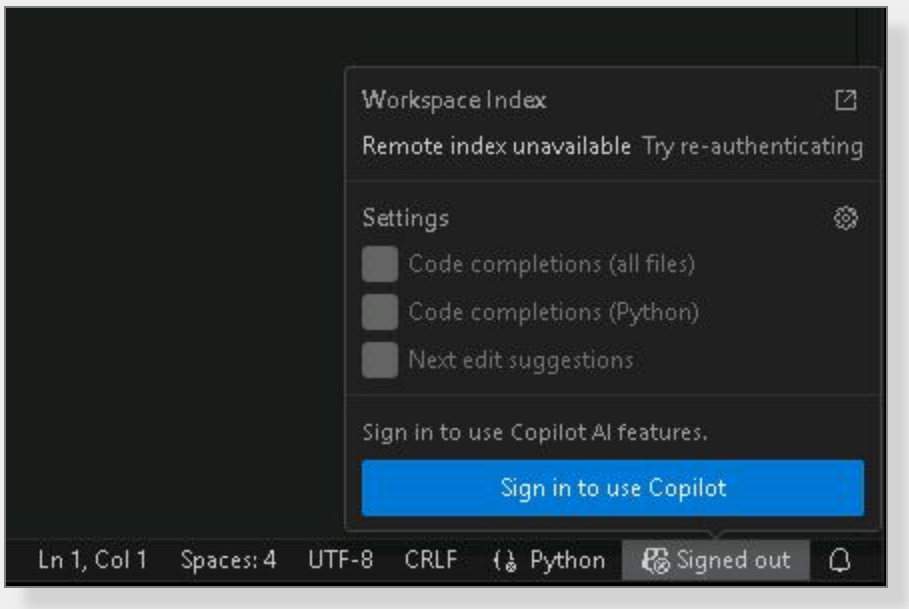
  GitHub Email Address

  GitHub Password

- Sign in to T Visual Studio Code with your GitHub account as T <GithubUN> by using T <GithubPW> as the password.

Expand this hint for guidance on signing in to GitHub in Visual Studio Code.

- On the Windows taskbar, in Search, enter T Visual Studio Code, and then select **Visual Studio Code**.
- In the bottom right-hand corner, select *Signed out*, select **Sign in to use Copilot**, and then sign in as T <GithubUN> by using T <GithubPW> as the password.



- Select **Continue**, and then select **Open** in the pop-up window.

You may close the *Welcome* tabs.

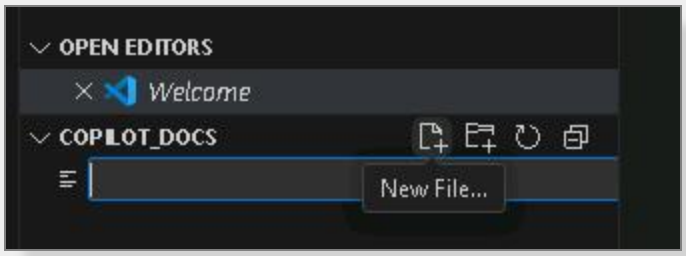
- Create the T copilot_docs project directory in the T D:\LabFiles folder, and then create a new file named T math_utils.py.

Expand this hint for guidance on creating the copilot_docs project directory and math_utils.py file.

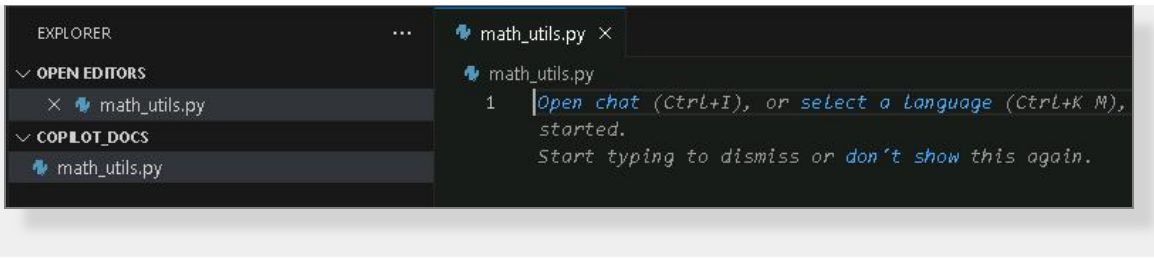
- In **Visual Studio Code**, select the **Terminal** menu, and then select **New Terminal**.
- Open a new terminal, and then change directories to the **D:\LabFiles** folder by using T cd D:\LabFiles.
- Create the T copilot_docs project directory by entering T mkdir copilot_docs, and then press **Enter**.

```
PS C:\Users\LabUser> cd D:LabFiles
PS D:\LabFiles> mkdir copilot_docs


    Directory: D:\LabFiles


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         1/20/2026   2:39 PM                copilot_docs


PS D:\LabFiles>
```

- Change to the working directory by entering T cd copilot_docs, and then press **Enter**.
- Enter T code ., and then press **Enter** to open the folder in Visual Studio Code.
- In the pop-up, *Do you trust the authors of the files in this folder?*, select **Yes, I trust the authors**.
- In the **Explorer** pane on the left, in the *COPILOT_DOCS* project directory, select the **New File** icon.



- Enter T math_utils.py , and then press **Enter** to create the file.



Want to learn more? Review the documentation on using the integrated terminal in Visual Studio Code.

When you open the folder, **Visual Studio Code** may ask *Do you trust the authors of the files in this folder?*. Select **Yes, I trust the authors**.

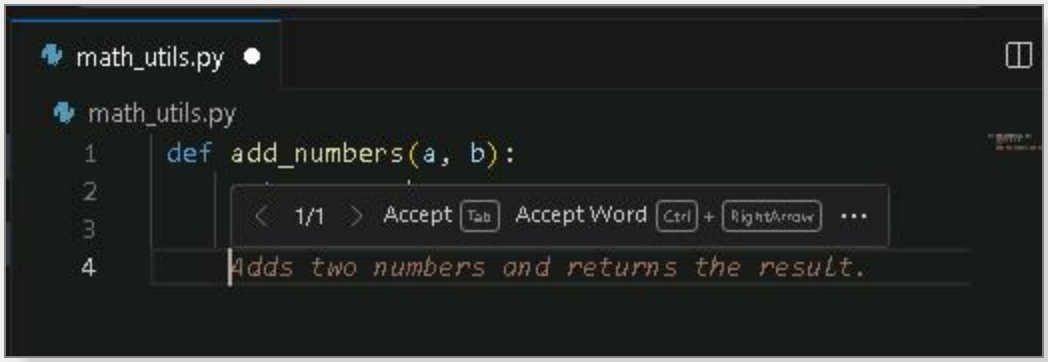
You may close any additional **Visual Studio Code** windows.

- Add the T add_numbers function and generate its docstring by using the following:

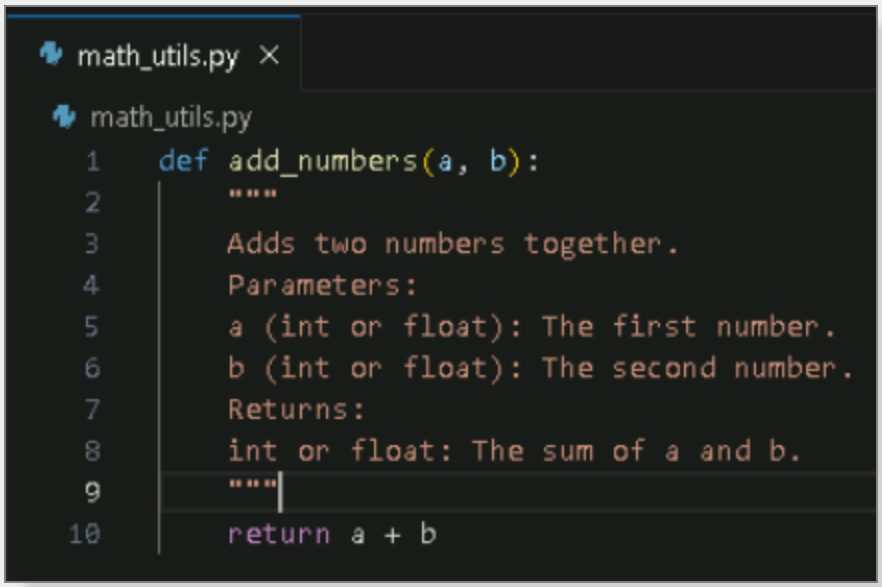
```Python
def add_numbers(a, b):
    return a + b
```

Expand this hint for guidance on adding the add_numbers function and generating its docstring.

- In the **math_utils.py** editor window, enter the Python code supplied above.
- Place your cursor on the line directly below the first line of **def add_numbers(a, b):**, enter T """, and then press **Enter**.
- Wait for the ghost text suggestion from Copilot to appear.



- Select the **Tab** key to accept the suggested docstring, and then press **Enter**.
- Continue selecting **Tab** and **Enter** until your script looks like the following:



- Select the **File** menu, and then select **Save**.

Want to learn more? Review the documentation on getting started with GitHub Copilot.

Copilot's ghost text suggestions appear as faded text directly in your editor. If a suggestion does not appear immediately, wait a moment or start entering the expected

text to give it more context.

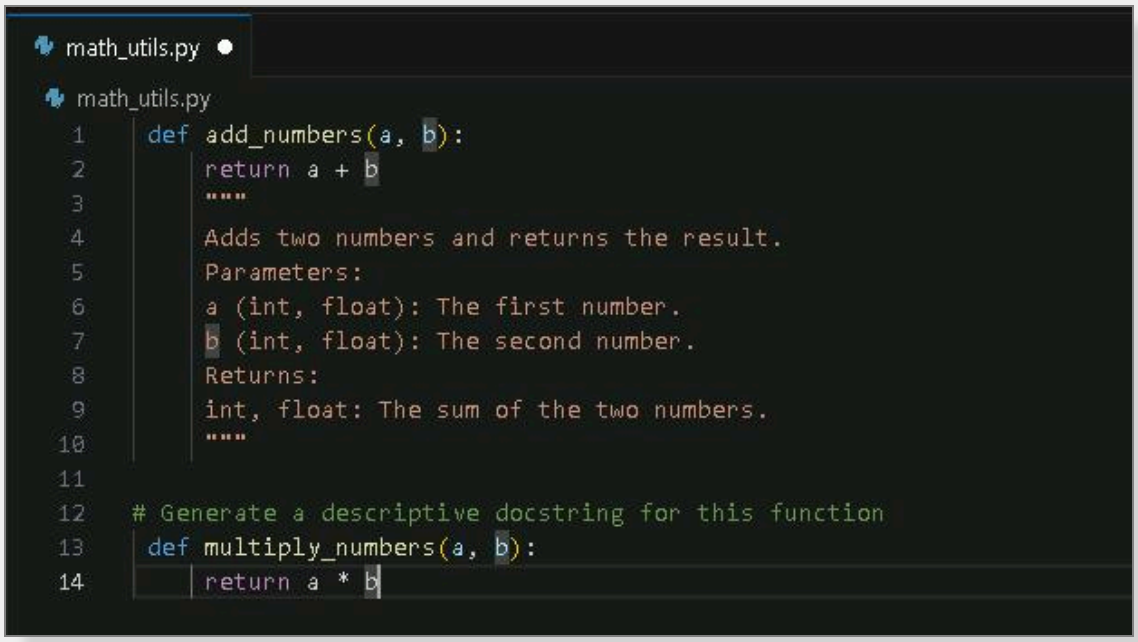
You can cycle through suggestions with **Alt+]** (or **Option+]**) and accept them by selecting the **Tab** key.

- Add the T multiply_numbers function and generate its docstring with a natural language prompt by using the prompt T # Generate a descriptive docstring for this function and the following Python code:

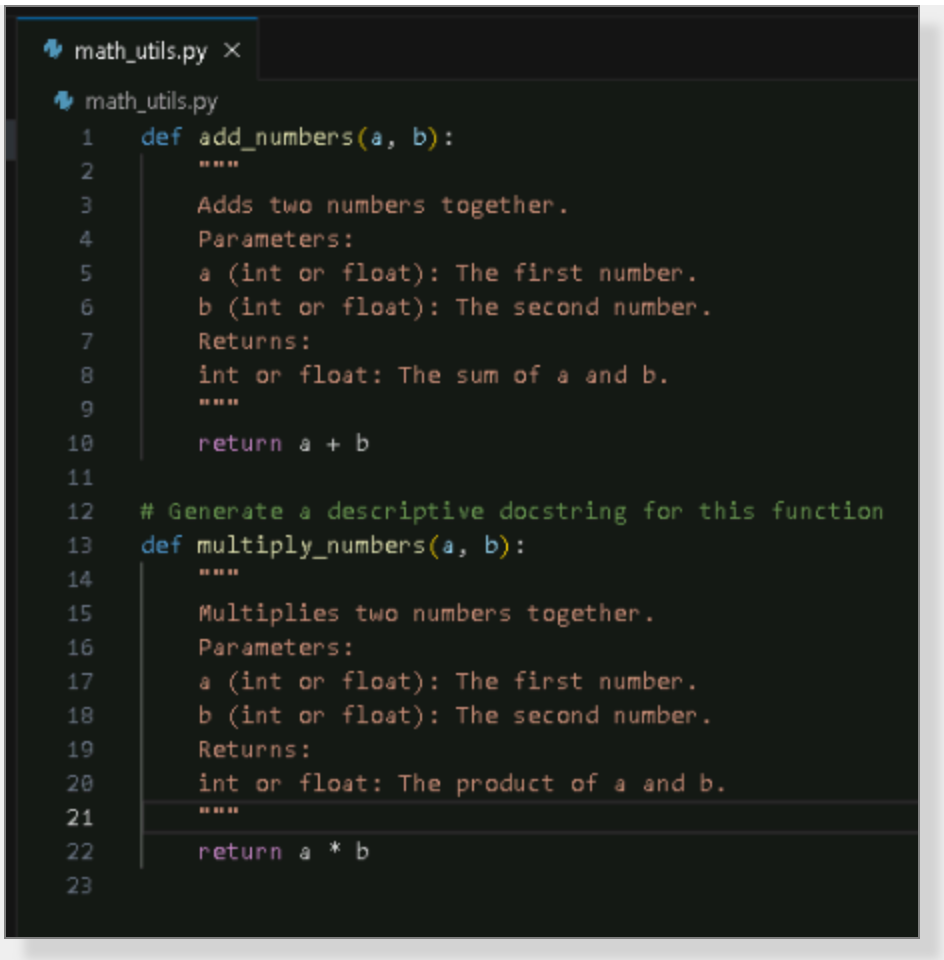
```Python
def multiply_numbers(a, b):
    return a * b
```

Expand this hint for guidance on adding the multiply_numbers function and generating its docstring with a natural language prompt.

- At the end of the **math_utils.py** file, add a few new lines, then add the Python code from above.
- On the line directly above the **def multiply_numbers(a, b):** line, enter the following comment: T # Generate a descriptive docstring for this function.



- Place your cursor on the line directly below the **multiply_numbers** function definition.
- Enter T """, and then wait for Copilot to suggest a multi-line docstring based on your prompt.
- Select the **Tab** key to accept the suggestion, and then press **Enter**.
- Continue selecting **Tab** and **Enter** until your script looks like the following:



- Select the **File** menu, and then select **Save**.

Want to learn more? Review the documentation on using GitHub Copilot Chat in your IDE.

Check your work

Verify

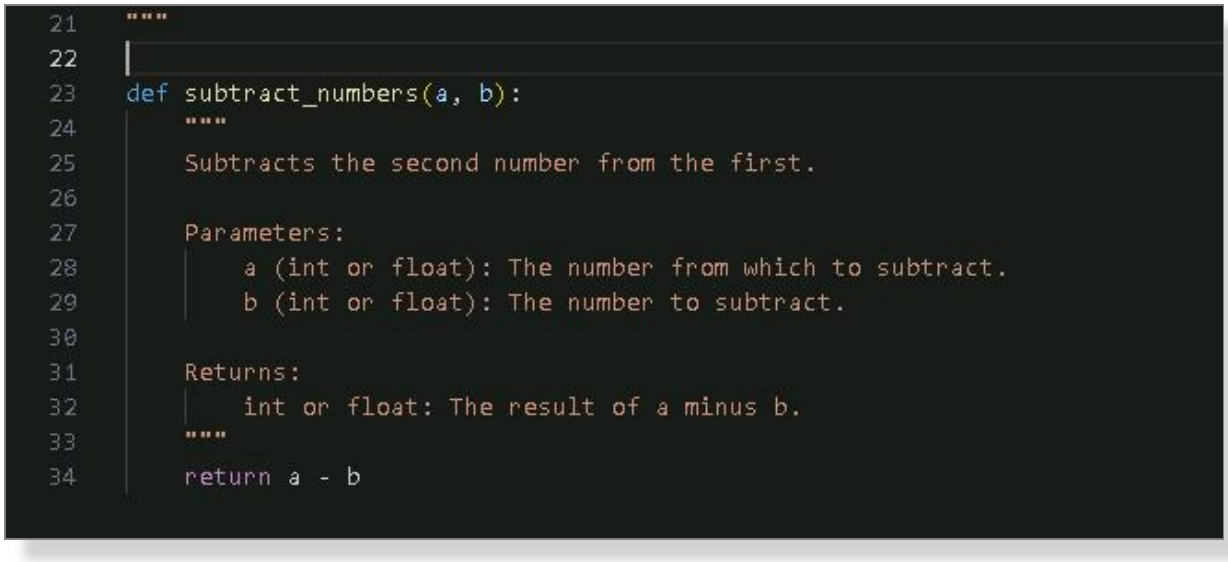
Add inline comments to a function with Copilot

Hints Enabled

No Yes

- Add the T subtract_numbers function to the **math_utils.py** file by using the following prompt:

```
Create a Python function named subtract_numbers that subtracts the second
number from the first.
Include a properly formatted triple-quoted docstring that explains:
- What the function does
- Parameters a and b (int or float)
- The return value and its type
Follow PEP 257 docstring conventions.
```



```
21  """
22
23  def subtract_numbers(a, b):
24      """
25      Subtracts the second number from the first.
26
27      Parameters:
28          a (int or float): The number from which to subtract.
29          b (int or float): The number to subtract.
30
31      Returns:
32          int or float: The result of a minus b.
33      """
34      return a - b
```

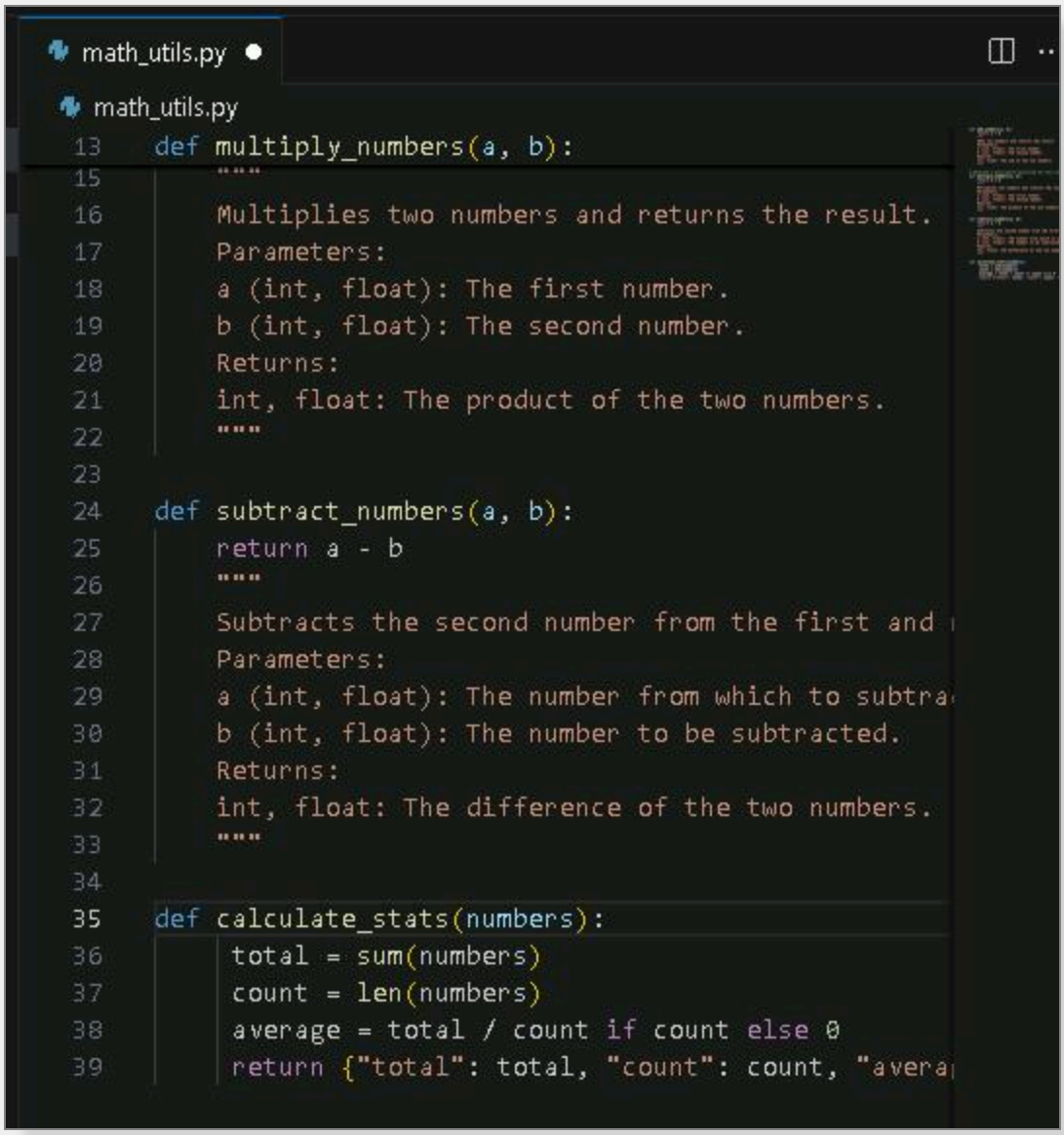
- Add the T calculate_stats function to the **math_utils.py** file by using the following Python code:

Python

```python
def calculate_stats(numbers):
    total = sum(numbers)
    count = len(numbers)
    average = total / count if count else 0
    return {"total": total, "count": count, "average": average}
```

Expand this hint for guidance on adding the calculate_stats function to the math_utils.py file.

- At the end of the **math_utils.py** file, enter the code supplied above.



Want to learn more? Review the documentation on Python functions.

- Add inline comments to the calculate_stats function with Copilot Chat by using the following comment and textbox prompts:
  - Comment:

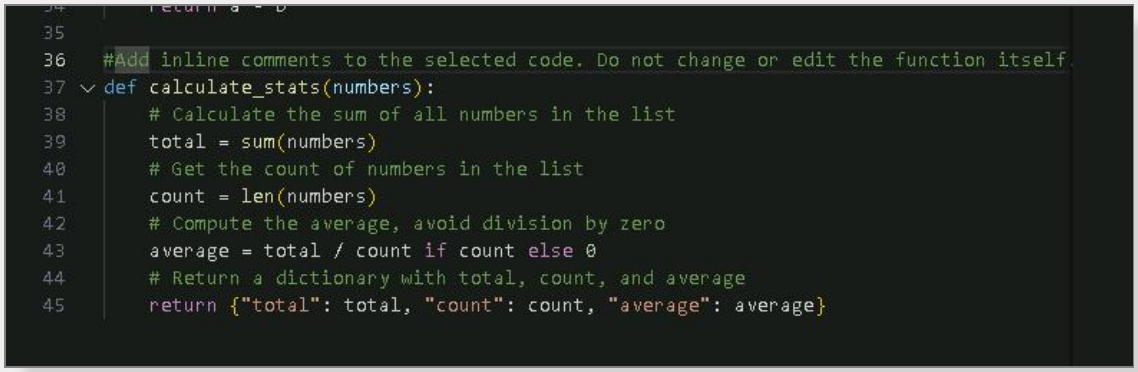
```
# Add inline comments to the selected code.
```

- Prompt:

```
# Add inline comments to the selected code. Do not change or edit the function itself.
```

Expand this hint for guidance on adding inline comments to the calculate_stats function with Copilot Chat.

- On the line directly above the **def calculate_stats(numbers):** line, enter the comment: T # Add inline comments to the selected code..
- Select the lines of code inside the **calculate_stats** function, and then select **Ctrl+I** to open the **GitHub Copilot Inline Chat** window.
- In the prompt box that appears, enter T # Add inline comments to the selected code. Do not change or edit the function itself. , and then press **Enter**.



- Review the changes proposed by Copilot.
- Select the **Accept** button to apply the inline comments.
- Select the **File** menu, and then select **Save**.

Want to learn more? Review the documentation on using Copilot Inline chat.

AI-generated code and comments should always be carefully reviewed.

While tools like GitHub Copilot are powerful, they can sometimes produce incorrect or suboptimal suggestions. Always verify the logic and accuracy of the generated content before accepting it into your codebase.

Check your work

Verify

Add a module docstring and verify documentation

Hints Enabled

No Yes

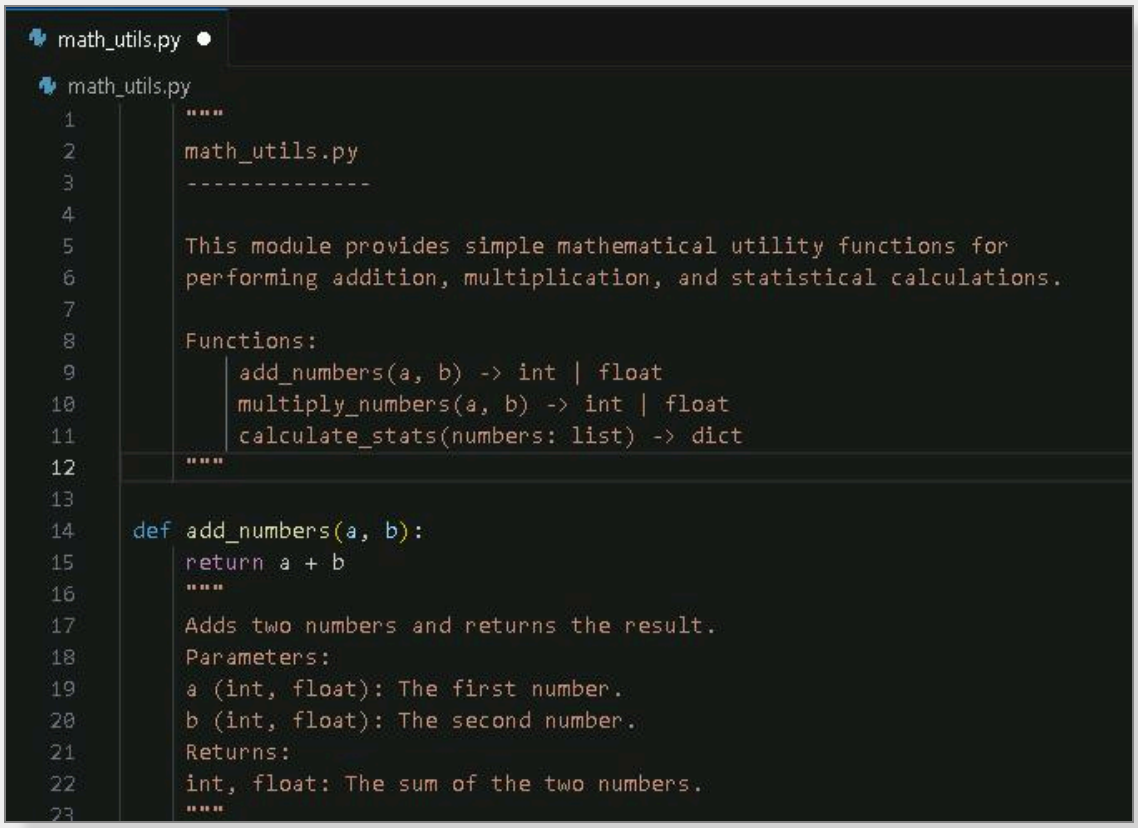
- Add a module-level docstring to the **math_utils.py** file by using the following:

Python

```
"""
math_utils.py
--------------

This module provides simple mathematical utility functions for
performing addition, multiplication, and statistical calculations.

Functions:
    add_numbers(a, b) -> int | float
    multiply_numbers(a, b) -> int | float
    calculate_stats(numbers: list) -> dict
"""
```

Expand this hint for guidance on adding a module-level docstring to the math_utils.py file.

- Navigate to the very top of the **math_utils.py** file, before any other code.
- On the first line, enter the multi-line docstring supplied above.



- Select the **File** menu, and then select **Save**.

Want to learn more? Review the documentation on Python Docstring Conventions (PEP 257).

- In **math_utils.py**, select **Ctrl+A**, and then select **Ctrl+C** to copy the entire script.
- In the **GitHub Copilot Chat** textbox to the right, enter T Fix all indentation issues, paste the entire script, and then select **Send**.

You can use **Ctrl+Shift+I** to open the GitHub Copilot Chat textbox.

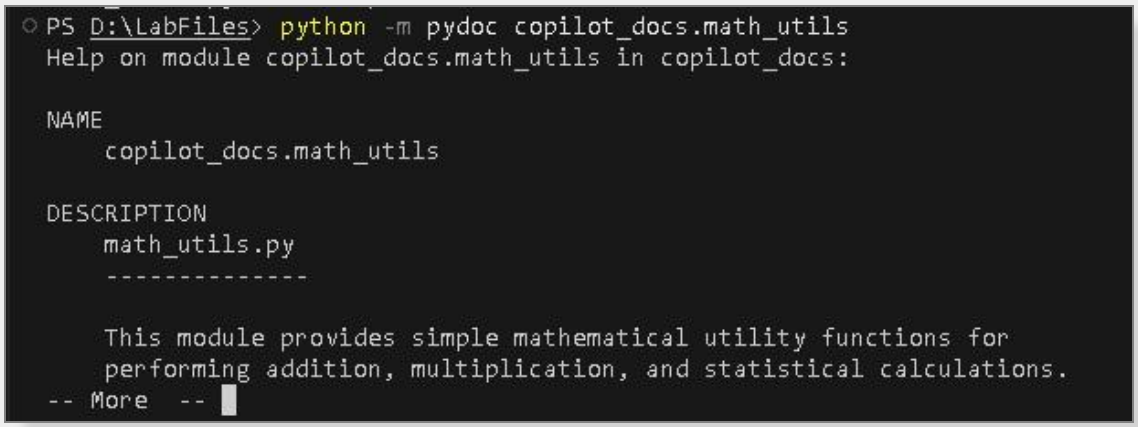
- Accept the changes and save the file.
- Verify the module documentation by entering the following into the **VS Code** integrated terminal:

Python

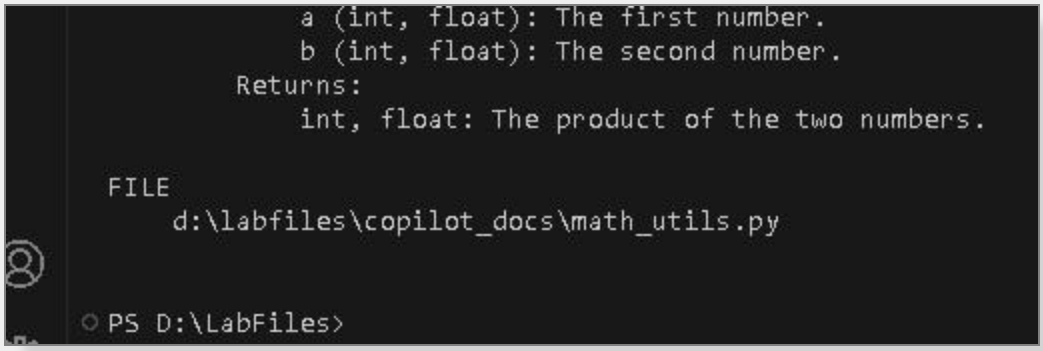
```
python -m pydoc copilot_docs.math_utils
```

Expand this hint for guidance on verifying the module documentation with the pydoc command.

- In Visual Studio Code, select the **Terminal** menu, and then select **New Terminal**.
- If the terminal prompt shows you are inside the **copilot_docs** directory, enter `cd D:\LabFiles`, and then press **Enter** to move to the parent directory.
- In the terminal, enter the command `python -m pydoc copilot_docs.math_utils`, and then press **Enter**.
- Review the output in the terminal to confirm the module docstring, function docstrings, and comments are displayed correctly.



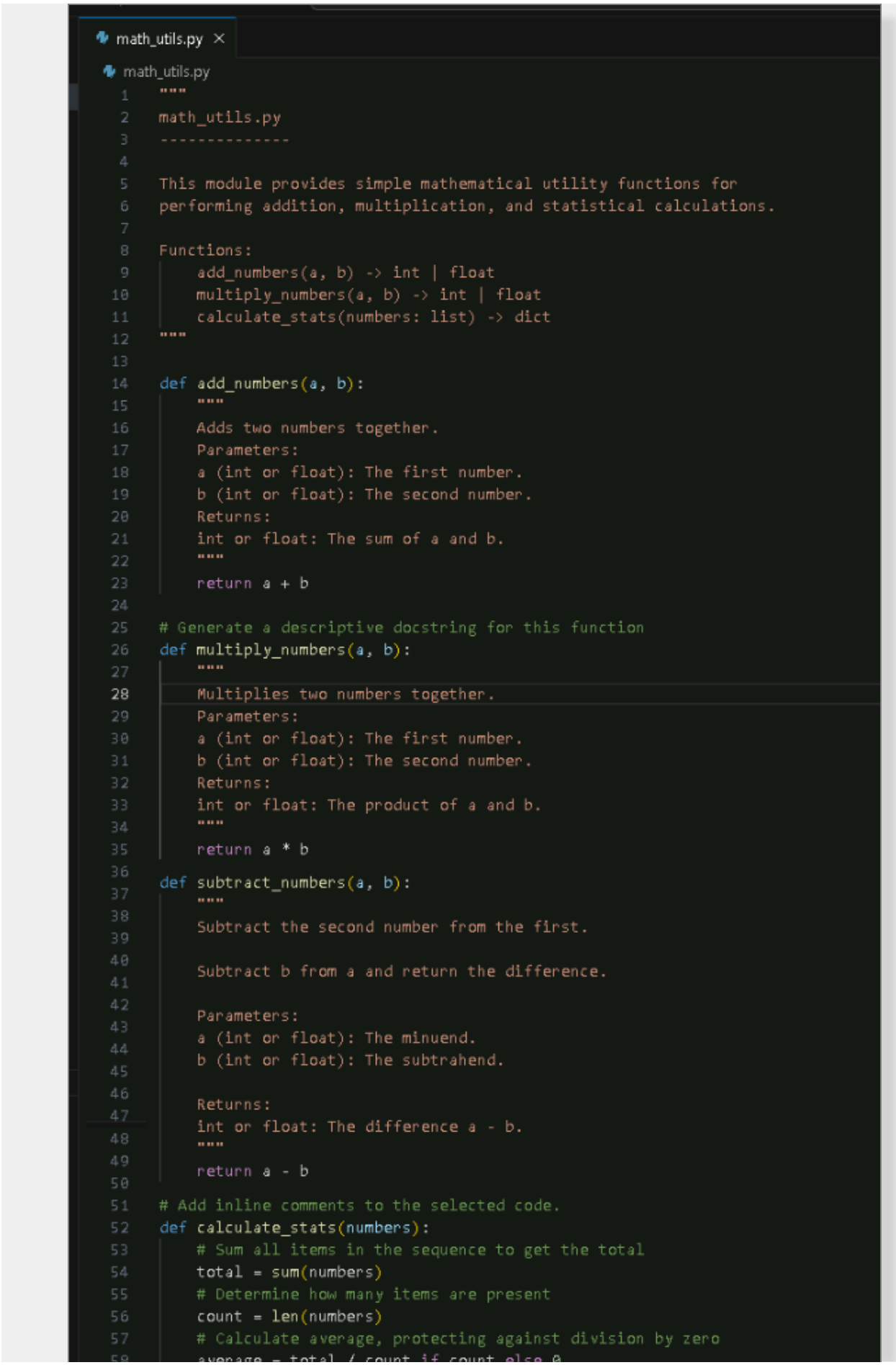
- Repeatedly press **Enter** (or press the **Spacebar**) until you return to the **PS D:\LabFiles>** prompt.



Want to learn more? Review the documentation on pydoc - documentation generator and online help system.

The finished script example:

Your script should look similar to the example below.


math_utils.py
math_utils.py
"""
math_utils.py

This module provides simple mathematical utility functions for
performing addition, multiplication, and statistical calculations.
Functions:
add_numbers(a, b) -> int | float
multiply_numbers(a, b) -> int | float
calculate_stats(numbers: list) -> dict
"""
def add_numbers(a, b):
"""
Adds two numbers together.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:
int or float: The sum of a and b.
"""
return a + b
# Generate a descriptive docstring for this function
def multiply_numbers(a, b):
"""
Multiplies two numbers together.
Parameters:
a (int or float): The first number.
b (int or float): The second number.
Returns:
int or float: The product of a and b.
"""
return a * b
def subtract_numbers(a, b):
"""
Subtract the second number from the first.
Subtract b from a and return the difference.
Parameters:
a (int or float): The minuend.
b (int or float): The subtrahend.
Returns:
int or float: The difference a - b.
"""
return a - b
# Add inline comments to the selected code.
def calculate_stats(numbers):
# Sum all items in the sequence to get the total
total = sum(numbers)
# Determine how many items are present
count = len(numbers)
# Calculate average, protecting against division by zero

```
    # Return results as a dictionary
    return {"total": total, "count": count, "average": average}
```

- Use the Python Interactive Shell to import and test a module to *D:\LabFiles* by using the following in a command prompt, and then call the **add_numbers** function using T add_numbers(2, 3).

```
from copilot_docs.math_utils import add_numbers, subtract_numbers, calculat
```

Expand this hint for guidance on using Python to test a module.

- Open a **Command Prompt** window by entering T cmd in the Windows taskbar Search.
- Change to the **D** drive by entering the following, and then press **Enter**:

```
D:
```

- Start the Python interactive shell by entering the following, and then press **Enter**:

```
python
```

- Verify that the **>>>** Python prompt appears.
- Import functions from the module by entering the command supplied above, and then press **Enter**.
- Call the **add_numbers** function to verify the import by entering the following, and then press **Enter**:

```
add_numbers(2, 3)
```

- Confirm that Python returns the expected result: **5**.

- Test the **subtract_numbers** function by entering the following, and then press **Enter**:

```
Python
subtract_numbers(50, 25)
```

- Confirm that Python returns the expected result: **25**.

Check your work

Verify

Summary

Congratulations, you have completed the **Add Comments and Docstrings by Using Copilot** Challenge Lab.

You have accomplished the following:

- Generated function docstrings with Copilot.
- Added inline comments to a function with Copilot.
- Added a module docstring and verified documentation.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

⚠ Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to Challenge Labs feedback.

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]
- Refactor Python Code by Using Copilot [Guided]

- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]